

GTreasury

Next-Generation Data & Workflow Orchestration Platform

Case Study Analysis, System Design & Presentation Q&A

Engineering Manager Case Study — Technical Presentation

Prepared by: John Jo | jjodev@gmail.com

September 16, 2026

Cloud Platform: Amazon Web Services (AWS)

Contents

1. Executive Summary	3
2. Context & Problem Statement	4
3. Requirements	5
4. Solution Strategy: A Two-Phase Approach.....	6
5. Phase 1 — Simple Single-Instance Solution (MVP / Pilot).....	7
6. Phase 2 — Enterprise-Grade Orchestration Platform.....	11
7. Technology Selection Rationale	16
8. Operational Excellence Strategy	21
9. Security & Compliance Considerations	25
10. Presentation Q&A — “The Challenge”	26
11. Delivery Roadmap & Milestones.....	31
12. Assumptions	32

1. Executive Summary

GTreasury's core value proposition is a single pane of glass over a customer's treasury and finance operations. That view depends on a large and growing number of scheduled jobs that move and transform data between banks, ERPs, and GTreasury's own systems, keeping information such as a customer's hundred-plus global bank accounts synchronized and current. The current implementation, built as a collection of scheduled jobs, has served the business well but is beginning to show its age as customer count, data volume, and workflow complexity all grow.

This document lays out how the platform can evolve into a purpose-built data and workflow orchestration system: a catalog of reusable components — both scripted and compiled — that any GTreasury engineering team can contribute, composed into multi-step workflows, with fault tolerance, elastic scalability, and resiliency treated as first-class design goals from day one, and with self-service recovery available to customers, Customer Support, and Professional Services when something goes wrong, rather than every recovery depending on Platform Operations or Engineering.

Recommended approach: deliver incrementally, not as a big-bang rewrite. Start with a simple, single-instance implementation that proves the workflow/component model with real customers in weeks rather than quarters, then evolve — without discarding what was built — into a highly available, elastically scalable, multi-region enterprise platform built natively on AWS managed services. At enterprise scale, the design also introduces an AI agentic automation layer — built on Amazon Bedrock Agents — that triages failed jobs, executes low-risk recovery actions autonomously, routes higher-risk actions to a one-click human approval, and helps contributing engineering teams scaffold new catalog components faster, directly extending the self-service recovery and developer-productivity goals called out in the case study.

This document covers, in order:

- The problem context, personas, and requirements drawn from the case study.
- The target architecture for a simple Phase 1 (single-instance) solution and the enterprise-grade Phase 2 solution, with diagrams.
- An AI agentic automation layer added to the Phase 2 enterprise solution, with its own architecture, guardrails, and governance.
- Explicit justification for every major technology choice, including the alternatives considered.
- The operational strategy across scalability, availability, resiliency, observability, and disaster recovery.
- Presentation-ready answers to every question posed in the case study's "The Challenge" section.
- A delivery roadmap, KPI framework, and the assumptions this analysis relies on.

2. Context & Problem Statement

GTreasury aggregates data across a customer's financial and ERP institutions so that, for example, a hundred bank accounts held globally can be seen and managed in one place. Today this is achieved through scheduled jobs that move files around and transform them from one format to another — for instance, pulling recent bank transactions from a bank's FTP server on a schedule, downloading the file, transforming it from the bank's proprietary format into a standard internal format, applying business rules such as calculations, storing the result in the database, and notifying subscribed parties once the job completes or fails.

Because customers' business operations are tightly coupled to these jobs completing correctly, scheduled jobs are one of the most critical components of the system: even a rare failure is an urgent priority to remediate. The most common failure modes called out in the case study are operation timeouts, connectivity issues, and corrupt files or erroneous records that block an entire batch from processing. Jobs also do more than move data — they send alerts and notifications, generate and email reports, update database state (driving workflow status, calculating derived values), perform clean-up and maintenance tasks, and can themselves be chained into a series of tasks that form a larger workflow.

The team's stated direction is to evolve this from a set of independent scheduled jobs into a genuine data and workflow orchestration platform, where fault tolerance, scalability, and resiliency are the top priorities, where the system scales elastically with load, where a catalog of reusable components contributed by any GTreasury engineering team can be composed into workflows for any orchestration or system-automation use case, and where — when something does fail — customers and Customer Support have self-service tools to recover, rather than depending on Platform Operations or Engineering for every incident.

2.1 Personas

- **End customer / user** — highly attuned to job execution because their own business operations depend on the outcome; wants to know when jobs complete and, on failure, to take corrective action themselves.
- **Professional Services** — configures these workflows when onboarding a customer; every customer's needs are unique, so they depend on the flexibility of the available components and the scheduling model.
- **Customer Support** — provides ongoing support and today acts as the bridge between the customer and Platform Operations whenever recovery is required.
- **Platform Operations** — owns day-to-day operations and systems administration, is accountable for a durable platform, and provides advanced technical recovery support.
- **Engineering** — builds and integrates both the scheduling platform itself and the schedulable components that run on it.

3. Requirements

3.1 Functional Requirements

- Define workflows as a composition of reusable tasks/components, chained into a DAG (directed acyclic graph) of steps.
- Components can pull and push data via API, SFTP/FTP, and AS2, transform it between formats, and apply business rules and calculations.
- Update database state as part of a job — driving workflow status, computing derived values, and similar automation.
- Send notifications and alerts, and generate and email reports, as first-class job actions, not afterthoughts.
- Run clean-up and maintenance tasks on the same scheduling substrate as data-movement jobs.
- Support both scripted (interpreted) and compiled-language components in the same catalog.
- Allow any GTreasury engineering team to contribute a new component to the shared catalog without Engineering becoming a bottleneck.
- Give customers and Customer Support self-service visibility into job/workflow status and a supported way to recover from a failure — without always escalating to Platform Operations or Engineering.

3.2 Non-Functional Requirements

The case study calls out fault tolerance, scalability, and resiliency as the top priorities, with availability, observability, and disaster recovery treated as equally critical to the platform's operational excellence. This analysis also treats multi-tenancy/tenant isolation, security, and developer productivity as implicit requirements given the multi-team contribution model and the sensitivity of financial data.

Quality Attribute	Target / Design Intent
Fault tolerance & resiliency	Transient failures (timeouts, connectivity blips) self-heal via retries/backoff without human involvement; a single bad file/record cannot block an entire batch or tenant.
Scalability	Compute and messaging scale elastically with demand — including predictable seasonal spikes (e.g., month-end/quarter-end treasury close) — with no manual capacity pre-provisioning.
Availability	99.95%+ for the orchestration control plane and self-service portal, trending toward 99.99% for Tier-1 workloads as multi-region maturity increases.
Observability	Every workflow execution is traceable end-to-end with a single correlation ID across metrics, logs, and traces, surfaced per persona (Engineering, Platform Ops, Customer Support).
Disaster recovery	Defined, tiered RTO/RPO targets, backed by cross-region replication and infrastructure-as-code, tested via scheduled failover exercises rather than assumed.
Multi-tenancy / isolation	One tenant's volume or bad data cannot degrade another tenant's jobs ("noisy neighbor" protection).

Quality Attribute	Target / Design Intent
Security & compliance	Encryption in transit and at rest, least-privilege access per component, full audit trail, appropriate for financial data crossing many banking institutions.
Extensibility / developer productivity	A governed component catalog lets any engineering team ship a new capability without owning the orchestration engine itself.

4. Solution Strategy: A Two-Phase Approach

Rather than a single “big-bang” rebuild, this analysis recommends a walking-skeleton strategy: deliver a real, working version of the new architecture's core ideas — workflows as composed, reusable components; built-in retries; observability; notifications — on the smallest reasonable AWS footprint first (Phase 1), then harden and scale that same conceptual model, rather than a different one, into the enterprise design (Phase 2). Concretely, this means the workflow/component contracts, the AWS-native primitives introduced in Phase 1 (S3 for files, Secrets Manager for credentials, SNS for notifications, Transfer Family for bank connectivity), and the operational habits (observability from day one, notifications on every job) all carry forward unchanged into Phase 2. What changes between phases is the orchestration engine and execution layer underneath those contracts, and the amount of redundancy, automation, and self-service built around them.

This staged approach lets the team validate the workflow model against real customer scenarios quickly, start showing migration progress early, and de-risk the larger enterprise investment before committing to it — while never producing throwaway work, which matters directly to the capacity-planning and delivery questions addressed later in this document.

The same staging principle applies to the AI agentic automation layer introduced in section 6.6: it is deliberately a Phase 2 capability, not a Phase 1 one, because an agent is only as good as what it is grounded in. Phase 1 and the early part of Phase 2 generate the real incident history, runbooks, and component contracts that later become the Knowledge Base the agent retrieves from — the agentic layer is the direct beneficiary of having built the rest of the platform first, not a parallel, disconnected effort.

5. Phase 1 — Simple Single-Instance Solution (MVP / Pilot)

Goal: the fastest responsible path to a real, usable orchestration engine that a pilot set of customers and workflows can run on, on a minimal and inexpensive AWS footprint, used to validate the workflow and component model with actual production data before investing in the full enterprise build.

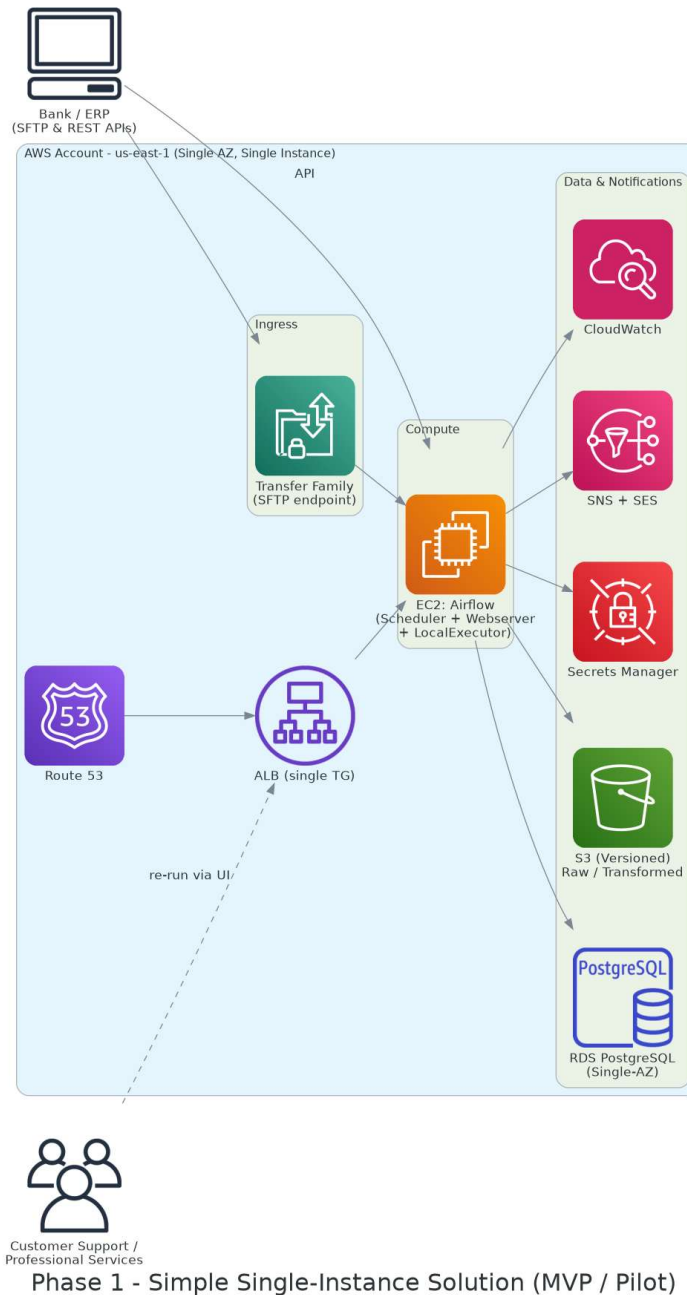


Figure 1 — Phase 1: simple, single-instance architecture (MVP / pilot).

5.1 How It Works

- **Orchestration engine:** a single EC2 instance runs Apache Airflow (Scheduler, Webserver, and LocalExecutor together). Workflows are authored as DAGs of reusable operators — the same composition model the platform is moving toward long-term.
- **Bank / ERP connectivity:** AWS Transfer Family exposes a managed SFTP endpoint so bank files can land without operating a file server; components also call bank/ERP REST APIs directly.
- **Metadata & state:** a single-AZ Amazon RDS for PostgreSQL instance holds Airflow's own metadata plus the platform's job configuration and run history.
- **Files:** a versioned Amazon S3 bucket holds every raw and transformed file, so nothing lives only on local disk.
- **Secrets:** AWS Secrets Manager holds every external bank/ERP credential from day one, so nothing sensitive is ever in code or config, and rotation is centralized before it's ever needed at scale.
- **Notifications & observability:** Amazon SNS/SES deliver job success/failure alerts and generated reports; Amazon CloudWatch captures logs, metrics, and alarms — observability is not deferred to Phase 2.
- **Access:** Route 53 and a single-target Application Load Balancer front the Airflow UI, giving Customer Support and Professional Services a real (if basic) window into job status and a manual “clear/retry task” recovery action — an early, limited step toward self-service.

5.2 Phase 1 Technology Stack

Layer	AWS Service / Tool	Purpose
Compute / Orchestration	EC2 + Apache Airflow (LocalExecutor)	Single-node workflow engine; DAG authoring and execution.
File transfer	AWS Transfer Family (SFTP)	Managed bank/ERP file exchange, no self-hosted FTP server.
Relational metadata	Amazon RDS for PostgreSQL (Single-AZ)	Airflow metadata plus job/workflow configuration and history.
Object storage	Amazon S3 (versioned)	Durable store for raw and transformed files.
Secrets	AWS Secrets Manager	Bank/ERP credentials, centrally managed from day one.
Notifications	Amazon SNS + SES	Job success/failure alerts and emailed reports.
Observability	Amazon CloudWatch	Logs, metrics, and alarms for the single instance and its jobs.
Access / DNS	Route 53 + Application Load Balancer	Stable URL and access path to the Airflow UI.

5.3 Known Limitations (Accepted, By Design)

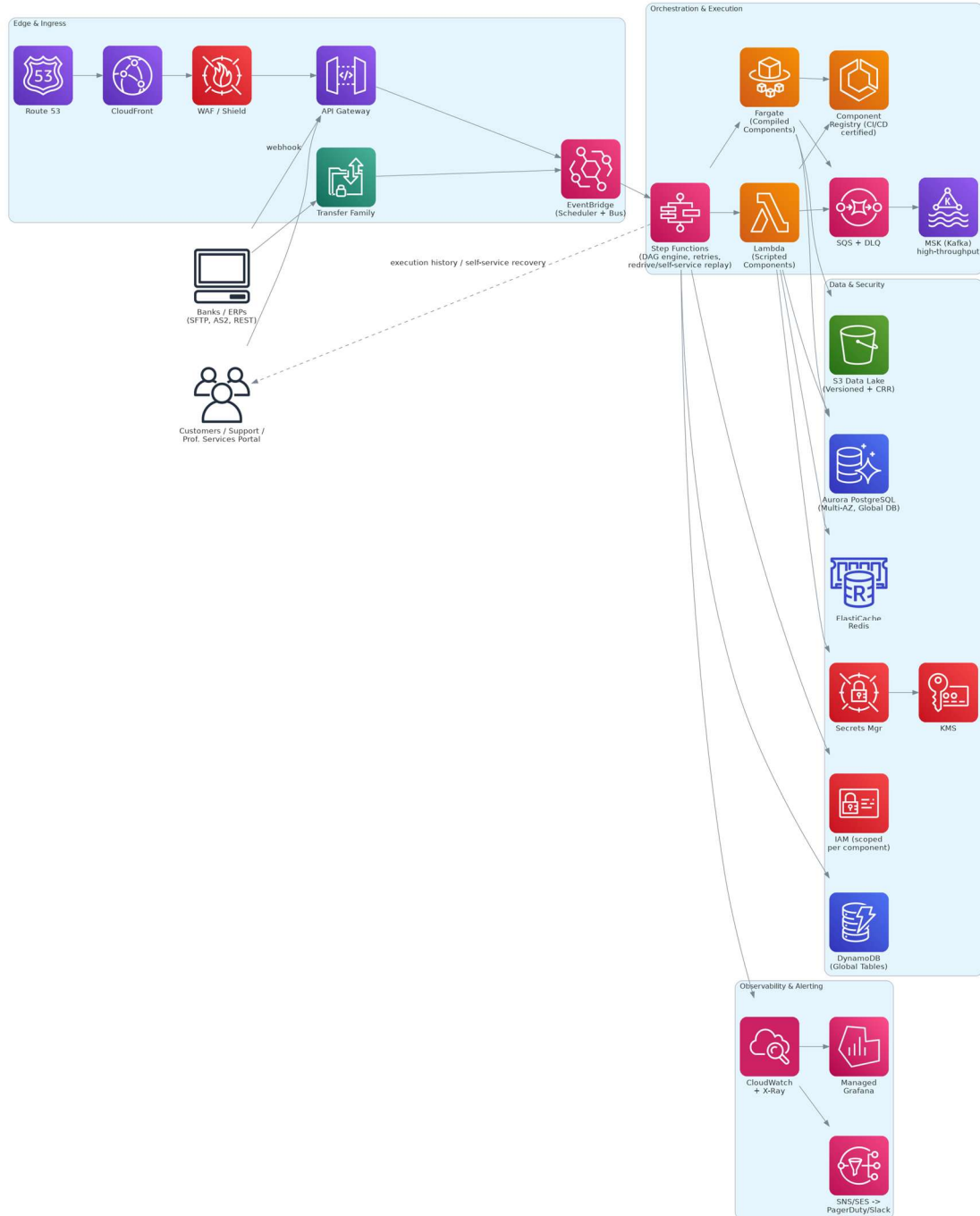
Phase 1 deliberately trades some of the case study's top priorities for speed to market; each limitation below is one Phase 2 is specifically designed to remove, and none of them block a safe pilot with a small, trusted set of customers and workflows:

- Single AZ, single instance — an instance failure causes downtime until EC2 auto-recovery or manual intervention completes; there is no automatic multi-AZ failover.
- LocalExecutor caps horizontal scale to one instance's CPU/memory — adequate for pilot volume, not the full customer base.

- No cross-region disaster recovery — recovery is via RDS snapshot restore and S3 durability, with RTO/RPO measured in hours, not minutes.
- No formal multi-tenant isolation — acceptable only while running a small, trusted pilot cohort rather than the full customer population.
- Self-service recovery is limited to an internal-facing “clear/retry task” action in the Airflow UI, not yet exposed directly to end customers.
- No AI agentic automation yet — failure triage and remediation in Phase 1 are entirely human-driven. The AI agentic automation layer (section 6.6) is deliberately introduced only at Phase 2, once there is enough real incident and runbook history for the agent's Knowledge Base to be grounded in actual GTreasury operational patterns rather than generic guesses.

6. Phase 2 — Enterprise-Grade Orchestration Platform

Goal: an elastic, multi-tenant, multi-region, serverless-first orchestration platform with a governed component catalog that any GTreasury engineering team can contribute to, and genuine self-service recovery for customers and Customer Support.



Phase 2 - Enterprise Data & Workflow Orchestration Platform (AWS)

Figure 2 — Phase 2: enterprise-grade data & workflow orchestration platform on AWS.

6.1 Orchestration Control Plane

AWS Step Functions replaces the single-node Airflow scheduler as the orchestration control plane. Each workflow is a versioned state machine; each state either invokes a catalog component (a Lambda function or a Fargate task) or performs native control flow (branching, parallel execution, wait, map-over-items). Retries, exponential backoff, and Catch/fallback behavior are declared per state, and every execution's full history — including exactly which state failed and why — is retained automatically. Step Functions' Redrive capability re-runs only the failed portion of a workflow, which becomes the technical backbone of the self-service recovery experience described in section 6.5.

6.2 Component Catalog & Governance

Every component — whether contributed by the platform team or another GTreasury engineering team — implements a standard Component Contract: a declared input schema, output schema, idempotency key, timeout, and retry policy. Scripted components (Python/Node) run on AWS Lambda; compiled or longer-running components (Java/.NET/Go, or anything exceeding Lambda's execution-time or package-size limits) run on ECS Fargate. A Component Registry — metadata in Aurora, versioned artifacts in Amazon ECR/S3 — is fronted by a CI/CD certification pipeline that runs contract tests and security scans before a new component version becomes available to compose into workflows. This is what lets “any GTreasury engineering team” contribute safely, without Engineering having to review every change by hand or becoming a release bottleneck.

6.3 Ingestion, Data Movement & Messaging

- AWS Transfer Family (SFTP/FTPS/AS2) and Amazon API Gateway (REST/webhooks) handle bank/ERP ingress at the edge, scaling automatically with connection volume.
- Amazon EventBridge is the single event bus: schedules, file-arrival events, and webhook events all normalize into EventBridge events that trigger Step Functions executions, decoupling what triggers a workflow from how it runs.
- Amazon SQS (with per-tenant queues and a dead-letter queue) buffers bursts ahead of component invocations and gives a clean, inspectable landing spot for the case study's “corrupt file / erroneous record” failure scenario.
- Amazon MSK (managed Kafka) is used selectively for very high-throughput or multi-consumer, replay-sensitive event streams — not the default path for most workflows.

6.4 Data & Security Layer

- Amazon Aurora PostgreSQL (Multi-AZ, Global Database) — workflow, component, and tenant configuration metadata.
- Amazon DynamoDB (on-demand, Global Tables) — high-volume execution state and idempotency keys.
- Amazon ElastiCache for Redis — short-lived distributed locks and hot-configuration caching.
- Amazon S3 (versioned, Cross-Region Replication, lifecycle policies) — the durable data lake and audit trail for every raw and transformed file.
- AWS Secrets Manager, AWS KMS, and IAM — every credential centrally managed and rotated, every component scoped to only the resources it needs.

6.5 Self-Service Recovery Experience

A portal (React/Next.js on S3 and CloudFront, backed by API Gateway and Lambda) reads directly from Step Functions' execution-history APIs. A Customer Support agent — and, once the permission model matures, the end

customer directly — can see the DAG for a specific run, see exactly which state failed and the structured error the component contract surfaced, and click “retry from failure” (a Redrive call) instead of opening a ticket to Platform Operations. This is the direct technical answer to the case study’s stated need: providing customers and Customer Support the capability to self-serve an easy recovery when a job fails.

6.6 AI Agentic Automation Layer

Phase 2 also introduces an agentic AI layer that goes beyond the static retries and Redrive-based recovery described above: rather than only giving a human a button to click, an AI agent reasons over the specific failure, decides what to do about it, and — within clearly scoped guardrails — either does it or proposes it for one-click approval. This is purpose-built on Amazon Bedrock Agents rather than a bespoke LLM pipeline, for the reasons detailed in section 7.18.

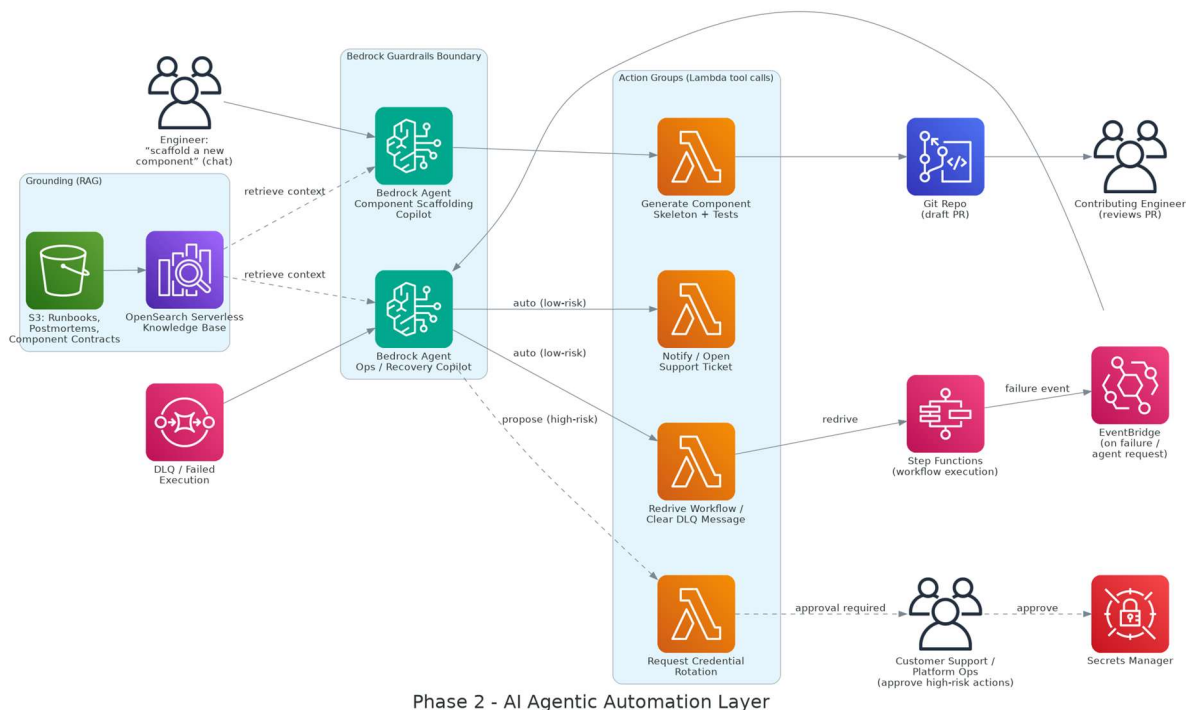


Figure 3 — Phase 2: AI agentic automation layer (Ops/Recovery Copilot and Component Scaffolding Copilot).

- **Ops / Recovery Copilot:** an EventBridge rule invokes a Bedrock Agent the moment a workflow execution fails or a message lands in a dead-letter queue. The agent is grounded via a Bedrock Knowledge Base (OpenSearch Serverless, indexed from S3) containing runbooks, past incident postmortems, and the Component Contract schema — so its diagnosis is retrieval-augmented against GTreasury’s own operational history, not a generic guess.
- **Tiered autonomy:** the agent can only act through a fixed set of Action Group Lambda tools, each as narrowly scoped as a catalog component’s own IAM role. Low-risk, reversible actions — redriving a failed Step Functions execution, clearing a transient DLQ message, notifying a subscriber, opening a support ticket — are executed autonomously. Anything higher-risk or irreversible — most notably requesting a bank/ERP credential rotation — is only ever proposed; a Customer Support or Platform Operations user must approve it with one click before it executes. The agent never has standing write access to financial data itself.

- **Guardrails for Amazon Bedrock** constrain the agent to its approved tool set and topics, blocking it from attempting anything outside the Action Group allowlist, and every agent decision and action is logged through the same CloudWatch/X-Ray pipeline as the rest of the platform, tagged distinctly in CloudTrail so an AI-initiated action is always visibly distinguishable from a human-initiated one.
- **Component Scaffolding Copilot:** a second agent, grounded on the same Knowledge Base, lets any contributing engineering team describe a new catalog component in natural language and receive a contract-compliant skeleton — handler stub, input/output schema, unit tests, and an IAM policy stub — as a draft pull request for a human engineer to review. This directly targets the “components contributed by any of GTreasury’s engineering teams” requirement and the explicit need to keep improving developer productivity alongside operational excellence (section 8).
- **Where the agent defers:** a failure the agent cannot confidently classify against its grounded context — for example, a file with records failing a business-rule validation in a way no runbook covers — is explicitly flagged for human review (Customer Support or Professional Services) rather than guessed at, consistent with keeping this layer additive to, not a replacement for, the human recovery path in section 6.5.

6.7 Phase 2 Technology Stack

Layer	AWS Service	Purpose
Orchestration	AWS Step Functions	Serverless workflow engine: DAG execution, retries, versioning, Redrive-based recovery.
Scripted components	AWS Lambda	Short, bursty, scripted task execution.
Compiled components	AWS Fargate (ECS)	Long-running or compiled-language task execution.
Component governance	Amazon ECR + CodePipeline/CodeBuild	Versioned artifacts and CI/CD certification for contributed components.
Event bus / scheduling	Amazon EventBridge	Normalizes schedules, file events, and webhooks into workflow triggers.
Ingress (files)	AWS Transfer Family	Managed SFTP/FTPS/AS2 for bank/ERP file exchange.
Ingress (API)	Amazon API Gateway	REST/webhook ingress and the self-service portal's API.
Buffering	Amazon SQS + DLQ	Backpressure, retry buffering, poison-message isolation.
High-throughput streaming	Amazon MSK (Kafka)	Selective use for high-volume, multi-consumer event streams.
Relational metadata	Amazon Aurora PostgreSQL	Workflow/component/tenant configuration, Multi-AZ + Global Database.
Execution state	Amazon DynamoDB	High-volume run state and idempotency keys, Global Tables.
Cache / locks	Amazon ElastiCache for Redis	Distributed locks and hot-configuration caching.
Object storage	Amazon S3	Data lake for raw/transformed files, versioned + cross-region replicated.

Layer	AWS Service	Purpose
Secrets & keys	AWS Secrets Manager + KMS	Credential storage/rotation and encryption at rest.
Identity	AWS IAM	Least-privilege, per-component scoped access.
Edge / CDN	CloudFront + WAF + Global Accelerator	Public entry point, web protection, fast regional failover.
Observability	CloudWatch, X-Ray, Managed Grafana	Metrics, logs, tracing, and persona-specific dashboards.
Notifications	Amazon SNS/SES + PagerDuty/Slack	Alerts, reports, and on-call escalation.
AI agentic automation	Amazon Bedrock Agents + Guardrails	Agent reasoning, tool-calling, and policy enforcement for Ops/Recovery and Scaffolding copilots.
Agent grounding (RAG)	Bedrock Knowledge Bases + OpenSearch Serverless	Retrieval over runbooks, postmortems, and Component Contracts.
Agent-generated code delivery	AWS CodeCommit (or equivalent Git host)	Draft pull requests from the Component Scaffolding Copilot.

7. Technology Selection Rationale

Every major technology choice above was made against at least one credible alternative. This section makes that reasoning explicit — what was chosen, what else was seriously considered, and why the chosen option won for this specific problem — so the trade-offs are visible rather than assumed.

7.1 Simple/MVP Orchestration Engine (Phase 1)

Chosen: Apache Airflow, self-hosted on a single EC2 instance (LocalExecutor)

Alternatives considered: AWS Step Functions from day one; plain cron + shell scripts (status quo); Amazon MWAA

Airflow's DAG-as-code model maps almost directly onto the “workflow of composed tasks” concept the platform is moving toward, so Phase 1's DAGs and operator code are reusable design input for Phase 2, not throwaway work. Its web UI immediately gives Customer Support something better than asking Engineering to check logs — an early, limited step toward self-service — without first building a custom portal. Standing up Step Functions for the pilot was rejected because most of its value (elastic serverless scaling, the multi-team component catalog, cross-region DR) isn't needed at pilot volume, and it would require building a portal immediately just to expose execution history, which Airflow provides for free. Continuing with plain cron/shell scripts was rejected because it perpetuates the exact fragility — no retries, no dependency model, no run history — that motivated this initiative in the first place.

7.2 Enterprise Workflow Orchestration Engine (Phase 2)

Chosen: AWS Step Functions

Alternatives considered: Amazon MWAA (managed Apache Airflow); Temporal (self-hosted on EKS, or Temporal Cloud); a custom-built scheduler

Step Functions is fully serverless, scaling automatically with concurrent executions with no fleet or capacity plan to manage — directly satisfying the case study's “scale elastically based on load” priority. Its native per-state Retry/Catch semantics, versioned state machines, and durable execution history with a Redrive API become the backbone of self-service recovery, and its deep native integration with 200+ AWS services (and any HTTP endpoint) reduces custom glue code for the “jobs do more than move data” requirement (alerts, reports, DB updates, clean-up). MWAA was a strong alternative — Airflow's Python-DAG model and huge operator ecosystem are attractive — but it requires managing a persistent environment (scheduler/worker capacity, version upgrades) and its replay UX is not naturally suited to non-engineer self-service the way Step Functions' Redrive-plus-console is. Temporal offers best-in-class long-running/stateful workflow semantics, but adds meaningful operational overhead (self-hosting on EKS, or a recurring SaaS cost via Temporal Cloud) not yet justified versus an AWS-native, serverless-first choice. A custom scheduler was rejected outright as undifferentiated heavy lifting.

7.3 Compute for Catalog Components

Chosen: AWS Lambda (scripted components) + AWS Fargate (compiled/long-running components)

Alternatives considered: EC2 Auto Scaling groups for all components; Amazon EKS for all components; Lambda-only

Splitting by workload shape avoids forcing every component into one runtime's limits. Lambda gives sub-second cold starts, pay-per-millisecond billing, and zero server management for the very common case of short, bursty, scripted transforms, but is bounded by a 15-minute execution limit and package size. Fargate removes that ceiling for compiled-language or long-running components while remaining serverless — no EC2 fleet to patch or scale by hand. EKS was considered for the whole execution layer for its portability, but rejected as the default because it reintroduces cluster-level operational overhead (node-group scaling, upgrades, multi-tenant hardening) that Fargate abstracts away; it remains an option later only if a component genuinely needs Kubernetes-specific

capability, which is not a current requirement. Lambda-only was rejected because some existing compiled components and large-file transforms will not fit Lambda's execution and package limits.

7.4 Relational / Control-Plane Database

Chosen: Amazon Aurora PostgreSQL (Multi-AZ, Global Database)

Alternatives considered: Amazon RDS for PostgreSQL; self-managed PostgreSQL on EC2; Aurora Serverless v2

Workflow, component, and tenant metadata is inherently relational, so a relational engine is the right base fit; Aurora was chosen over standard RDS specifically because its storage layer replicates across three AZs with faster failover (typically under 30 seconds) and its Global Database feature gives sub-second cross-region replication with one-click promotion, which more closely matches the DR RTO/RPO targets in section 10.5 than RDS cross-region read replicas. Self-managed PostgreSQL on EC2 was rejected outright as unnecessary operational burden for a workload with no unusual customization need. Aurora Serverless v2 is flagged as a viable cost-saving option while load is low and unpredictable during Phase 1/early Phase 2, with a plan to move to provisioned capacity once load is steady enough for reserved pricing to be more efficient.

7.5 High-Volume Execution State Store

Chosen: Amazon DynamoDB (on-demand capacity, Global Tables)

Alternatives considered: Using Aurora for execution state as well; Amazon ElastiCache alone; MongoDB / Amazon DocumentDB

Execution state and idempotency keys are extremely high-write, simple key-based lookups (by execution ID) needing single-digit-millisecond latency at high concurrency during batch windows — exactly DynamoDB's design target — and on-demand capacity means never pre-provisioning throughput for seasonal spikes. Reusing Aurora for this too was considered for simplicity, but rejected because mixing a bursty, very-high-write pattern into the same store as tenant/workflow configuration risks contention on the store the whole control plane depends on. DynamoDB Global Tables also replicate execution state active-active across regions with no manual promotion step, directly serving the DR strategy in a way a single-region store would not.

7.6 Caching / Distributed Locking

Chosen: Amazon ElastiCache for Redis

Alternatives considered: DynamoDB conditional writes only; Amazon MemoryDB

Redis gives sub-millisecond in-memory access for short-lived locks (preventing two concurrent executions from double-processing the same bank file) and hot-configuration caching, both faster and cheaper at high call volume than routing every lock check through DynamoDB. DynamoDB conditional writes remain the mechanism for durable idempotency keys, where losing the record would be a correctness problem — each store is used for what it is best at rather than forcing one store to do both jobs. MemoryDB (Redis-compatible with multi-AZ durability) is noted as a future upgrade path if a lock/cache workload ever needs durability guarantees ElastiCache doesn't provide, but isn't justified as a starting point given current access patterns tolerate cache loss.

7.7 File / Object Storage

Chosen: Amazon S3 (versioned, lifecycle policies, Cross-Region Replication)

Alternatives considered: Amazon EFS; EBS-backed storage on EC2

Every raw and transformed file is naturally an immutable object with a clear key (tenant/workflow/run) — exactly S3's model — and S3 provides effectively unlimited scale, eleven-nines durability, native versioning (protecting against a bad transform overwriting good data), lifecycle rules to control cost as files age, and native Cross-Region Replication used directly in the DR design. EFS/EBS remain in play only as ephemeral, local scratch space during a

single component's execution when a POSIX filesystem is genuinely needed mid-processing; S3 stays the durable system of record before and after, rather than making a POSIX filesystem the durable store.

7.8 File Transfer / Bank & ERP Ingestion

Chosen: AWS Transfer Family (SFTP / FTPS / AS2)

Alternatives considered: Self-hosted SFTP server on EC2; third-party managed file transfer (MFT) SaaS

Banks and ERPs overwhelmingly standardize on SFTP/FTPS/AS2, and Transfer Family gives a fully managed, auto-scaling endpoint for those protocols backed directly by S3 — removing an entire historical failure category (“our FTP server ran out of disk or couldn't handle concurrent connections”) without operating any servers. A self-hosted SFTP server was rejected as undifferentiated heavy lifting a managed service already solves well. A third-party MFT SaaS product was rejected to avoid an external vendor dependency and an extra data hop for data that can stay natively inside the AWS account and VPC boundary.

7.9 API & Webhook Ingress

Chosen: Amazon API Gateway

Alternatives considered: Application Load Balancer in front of containers; AWS AppSync

API Gateway gives managed throttling, request validation, and per-tenant API-key/usage-plan support, with native integration into Lambda and Step Functions and no infrastructure to run — a good fit for the mostly REST/webhook integration pattern with banks, ERPs, and the self-service portal. An ALB-fronted container service was considered for more low-level control, but judged unnecessary complexity versus API Gateway's built-in throttling and auth integration here. AppSync (GraphQL) was not a fit since the integrations in question are REST/event-driven, not graph-query-shaped.

7.10 Event Bus / Scheduling Trigger

Chosen: Amazon EventBridge

Alternatives considered: Amazon SNS as the primary trigger bus; scheduling built directly into Step Functions only

EventBridge lets every trigger type — a cron-like schedule, an S3 file-arrival event, a webhook-originated event — normalize into one event format that fans out to Step Functions, decoupling what starts a workflow from how the workflow runs and making it trivial to add new trigger types later. SNS remains in the design for simple fan-out notifications but lacks EventBridge's content-based routing and native scheduler, so it is not a good fit as the primary trigger bus. Relying only on Step Functions' own schedule feature was rejected because it would tie every non-schedule trigger (file arrival, webhook) into bespoke glue code instead of one consistent eventing model.

7.11 Buffering Between Trigger and Execution

Chosen: Amazon SQS (with dead-letter queues)

Alternatives considered: Direct synchronous invocation; Amazon Kinesis Data Streams

SQS decouples the rate work arrives at from the rate it is safely processed, so a burst — many customers' files landing at month-end, for example — is queued and smoothed rather than causing throttling or dropped work; its native dead-letter queue gives a clean, inspectable landing spot for the case study's “corrupt file / erroneous record” failure scenario. Direct synchronous invocation was rejected because it offers no buffering at all. Kinesis was considered for its ordering and replay guarantees, but judged to add shard-management and checkpointing complexity beyond what a queue-and-retry pattern needs; it is the right tool specifically for the selective high-throughput streaming case below.

7.12 High-Throughput Event Streaming (selective use)

Chosen: Amazon MSK (managed Kafka)*Alternatives considered: Amazon Kinesis Data Streams; SNS/SQS fan-out only, for every workflow*

MSK is introduced only for the subset of workflows with very high event volume, or where several independent internal consumers each need their own durable, replayable read position on the same stream (for example, ledger-style event sourcing feeding multiple downstream systems) — a pattern Kafka's consumer-group and replay model handles more naturally than SQS's competing-consumer model. Kinesis is a close technical alternative and remains an option; MSK/Kafka is preferred where teams already have Kafka expertise or existing Kafka-based systems to integrate with. For the common case, SNS/SQS alone is deliberately kept as the default to avoid running Kafka where it is not needed.

7.13 Secrets & Key Management**Chosen: AWS Secrets Manager + AWS KMS***Alternatives considered: AWS Systems Manager Parameter Store; HashiCorp Vault*

Secrets Manager provides native automatic rotation for database and API credentials — critical given how many bank/ERP credentials this system holds — and fine-grained IAM-based access per component; Parameter Store supports secrets but requires more custom tooling for rotation. KMS customer-managed keys back encryption at rest across S3, Aurora, DynamoDB, and Secrets Manager with a full CloudTrail audit trail. Vault was considered for its strong multi-cloud secret-leasing model, but set aside to stay AWS-native and avoid another system to run and patch given the workload is single-cloud today.

7.14 Content Delivery / Public Entry Point**Chosen: Amazon CloudFront + AWS WAF + AWS Global Accelerator (for DR failover)***Alternatives considered: Application Load Balancer only, with no CDN; a third-party CDN/WAF*

CloudFront terminates TLS close to users and caches the self-service portal's static assets; WAF in front of it blocks common web exploits and can rate-limit abusive clients before they reach the API. Global Accelerator is used specifically for DR failover because it operates at the network layer using Anycast IPs and fails over in seconds, versus relying solely on DNS TTL expiry for Route 53 failover, which can take minutes and is at the mercy of resolver caching. A third-party CDN/WAF was rejected to avoid a second vendor relationship and extra network hop when the AWS-native stack already meets the requirement and integrates natively with Shield, GuardDuty, and Security Hub.

7.15 Observability Stack**Chosen: Amazon CloudWatch + AWS X-Ray + Amazon Managed Grafana***Alternatives considered: Datadog or New Relic (third-party SaaS); self-hosted Prometheus/Grafana/Jaeger*

CloudWatch and X-Ray are natively wired into every AWS service used elsewhere in the design (Step Functions, Lambda, Fargate, Aurora, DynamoDB) with no agent to deploy, minimizing time-to-first-dashboard; Managed Grafana adds flexible, persona-specific dashboards without operating a Grafana server. Datadog/New Relic offer a more polished unified UX and vendor-neutral portability if GTreasury ever becomes multi-cloud, and are flagged as a reasonable future upgrade once observability needs outgrow native tooling; they were not chosen initially mainly on cost-at-scale and to avoid a third-party data-egress dependency before it is proven necessary. Self-hosting Prometheus/Grafana/Jaeger was rejected because it reintroduces the operational burden of running observability infrastructure directly.

7.16 Infrastructure as Code & CI/CD**Chosen: AWS CDK (or Terraform) + CodePipeline/CodeBuild (or GitHub Actions) + Amazon ECR**

Alternatives considered: Manually managed (“ClickOps”) infrastructure; hand-written AWS CloudFormation

Multi-region parity for DR and a multi-team component-contribution model both require infrastructure and component packaging to be reproducible and reviewable, which only IaC and pipeline-driven delivery provide; manually managed infrastructure was rejected outright as incompatible with both requirements. CDK (or Terraform, matching whichever is GTreasury's existing standard) is preferred over hand-written CloudFormation for the readability and reuse of defining infrastructure in a general-purpose language, letting the Component Contract itself be expressed as a reusable construct that every contributing team consumes rather than copy-pasting configuration.

7.17 Multi-Region Disaster Recovery Pattern

Chosen: Warm standby (Tier 1 and Tier 2 workloads); backup-and-restore (Tier 3)

Alternatives considered: Pilot light for all tiers; full active-active multi-region for all tiers

Warm standby gives a strong RTO (single-digit minutes) without paying for full duplicate production capacity at all times, the right balance for a treasury-operations platform where jobs are time-sensitive (bank cut-offs) but where active-active's added cost and complexity (bi-directional conflict resolution, doubled run-rate) is not yet justified. Pilot light was considered as a cheaper option but rejected for Tier 1 specifically because scaling compute up from zero adds minutes to RTO that a “top priority” resiliency requirement should not have to accept; it remains the right, more cost-efficient choice for Tier 3 (internal/reporting) workloads via backup-and-restore instead.

7.18 AI Agentic Automation

Chosen: Amazon Bedrock Agents + Bedrock Knowledge Bases (OpenSearch Serverless) + Guardrails for Amazon Bedrock

Alternatives considered: A self-hosted LLM-orchestration layer (e.g., LangChain/LangGraph on Fargate calling a model API directly); a third-party agent-platform SaaS; a rules-engine-only approach with no LLM; a fully autonomous agent with no human-approval tier

Bedrock Agents natively handles the reasoning-plus-tool-calling loop, session state, and prompt orchestration, and Bedrock Guardrails gives a managed, auditable way to constrain what the agent may do and discuss — both are undifferentiated heavy lifting that would otherwise have to be hand-built and continuously security-reviewed. Staying within AWS also means the agent's actions flow through the same IAM, CloudTrail, VPC, and KMS boundaries already governing the rest of the platform, rather than standing up a new, separately-secured execution environment for a system with standing permission to act on financial workflows. A self-hosted LangChain/LangGraph layer remains more flexible and model-agnostic, and was set aside for the initial design only to avoid owning the operational burden of a bespoke agent runtime (session management, guardrail enforcement, tool-call retries) before the simpler managed option is proven insufficient. A third-party agent SaaS platform was rejected for the same data-boundary reason as the observability discussion in section 7.15 — it would mean financial execution context leaving the AWS account boundary to be reasoned over. A rules-engine-only approach was rejected because it cannot generalize beyond failure patterns someone already thought to encode, which is exactly the gap this layer exists to close. A fully autonomous agent with no human-approval tier was rejected outright given the financial-data context: every agent action is scoped to an explicit low-risk, reversible allowlist, with anything else routed to a human for one-click approval, per section 6.6.

8. Operational Excellence Strategy

Availability, resiliency, scalability, observability, and disaster recovery are treated as a single, connected strategy rather than five independent checklists, because a weakness in one undermines the others — for example, resiliency features are worthless if observability cannot detect that they fired.

8.1 Scalability

- Serverless-first compute (Lambda, Fargate, Step Functions) scales automatically with concurrent executions, so there is no fleet to pre-provision for predictable seasonal peaks such as month-end or quarter-end treasury close.
- DynamoDB on-demand capacity and Aurora Serverless v2 / read replicas absorb read and write bursts without manual resharding or capacity planning.
- SQS decouples triggers from execution, so bursts are queued rather than dropped or causing cascading failures; consumer concurrency (Lambda reserved/provisioned concurrency, Fargate target-tracking on queue depth) scales to match queue backlog.
- Tenant-aware partitioning gives high-volume tenants dedicated queues and concurrency allocations — a noisy-neighbor bulkhead so no single customer's volume can starve another's, directly relevant at the case study's "hundred bank accounts globally" scale.
- Every component must demonstrate 3x its highest observed 30-day peak load in staging as a condition of certification into the catalog, building headroom verification into the release process itself rather than discovering limits in production.

8.2 Availability

- Target: 99.95% for the orchestration control plane and self-service portal (roughly ≤ 22 minutes of unplanned downtime per month), with 99.99% as the aspirational target for Tier-1 customers once multi-region maturity increases.
- Multi-AZ by default for every stateful component: Aurora Multi-AZ, DynamoDB's inherent multi-AZ design, Fargate tasks spread across at least two AZs, and per-AZ NAT gateways.
- Route 53 health checks with failover routing, and Global Accelerator at the public entry point, shed a failing AZ or region in seconds rather than the minutes a DNS TTL expiry would otherwise take.
- Self-healing by default: ECS/Fargate's service scheduler and EC2 auto-recovery replace unhealthy tasks/instances automatically, so common failures resolve without paging a human.

8.3 Resiliency

- Every workflow step declares Retry (exponential backoff with jitter) and Catch (fallback state) as native Step Functions behavior, so transient failures — the case study's "operations timing out" and "connectivity issues" — resolve without human involvement.
- Dead-letter queues on every SQS queue and async Lambda invocation, with a runbook'd (and, over time, self-service) redrive path for poison messages — the direct answer to "corrupt files or erroneous records that block a batch of data from being processed."
- Idempotency keys, enforced via DynamoDB conditional writes, on every component so retries and redrives are always safe to run twice — essential when the data in motion is financial.

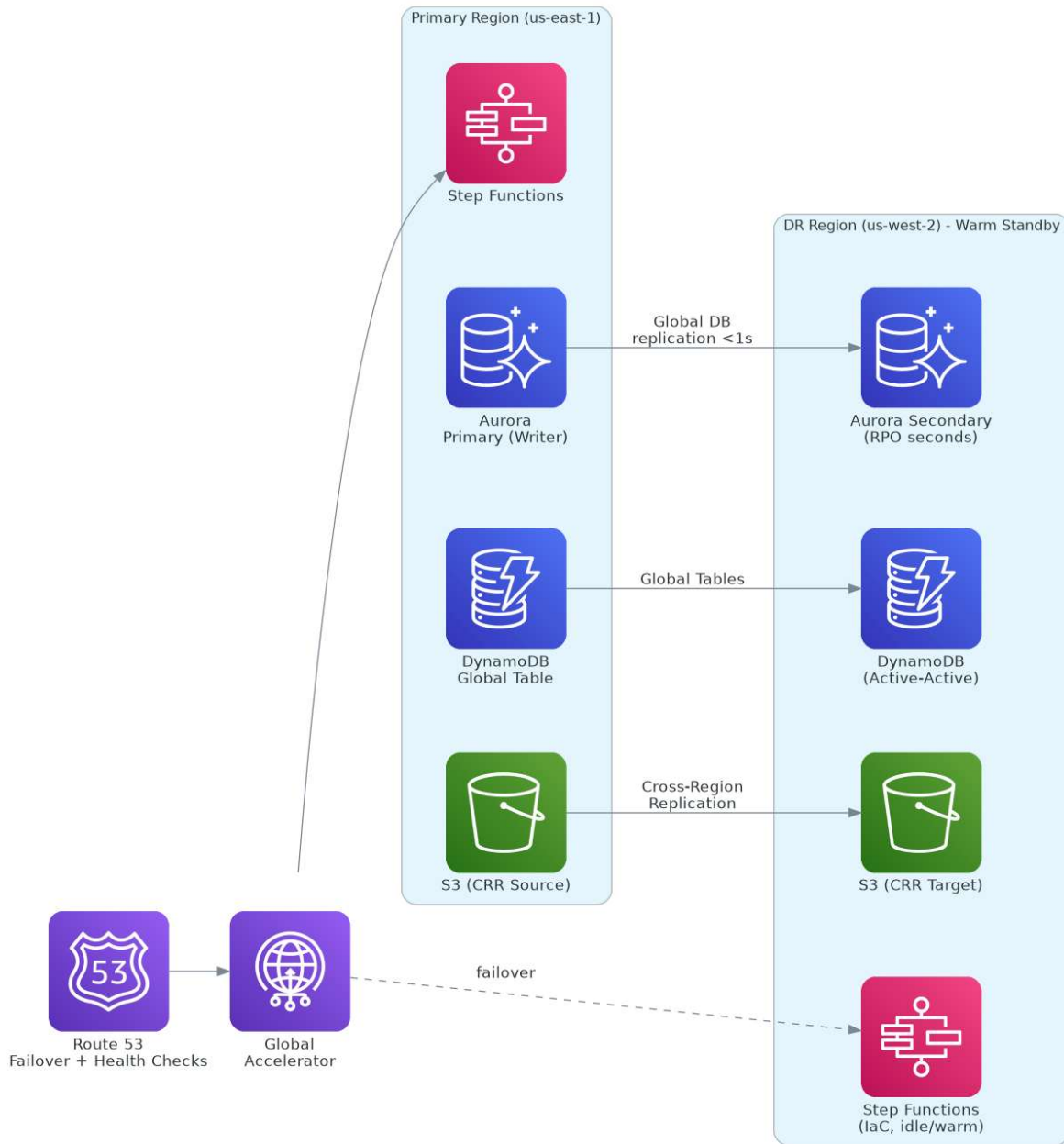
- Circuit breakers around outbound calls to bank/ERP endpoints open after a run of consecutive failures to a given institution, degrading gracefully instead of retry-storming a struggling external system.
- Bulkheads: per-tenant and per-external-integration concurrency limits so one bad actor — a bank outage, a malformed customer file — cannot exhaust capacity shared by everyone else.
- Scheduled AWS Fault Injection Simulator game days (killing an AZ, injecting Lambda throttling, forcing an Aurora failover) run quarterly against a staging environment that mirrors production topology, so resiliency claims are exercised, not assumed.

8.4 Observability

Three pillars, all correlated by a single execution/trace ID generated at workflow start and propagated through every component invocation:

- **Metrics** — CloudWatch custom metrics per workflow, component, and tenant (success rate, duration, retry count) feed CloudWatch Dashboards and Alarms, and Amazon Managed Grafana for persona-specific “mission control” views: Platform Operations sees infrastructure health, Customer Support sees per-customer job health, Engineering sees per-component health.
- **Logs** — structured JSON logs from every component ship to CloudWatch Logs, with a subscription filter into OpenSearch (or a company-standard log platform) for full-text search and correlation across a failed run.
- **Traces** — AWS X-Ray (or OpenTelemetry for cloud-portability) instrumented at every Step Functions/Lambda/Fargate boundary, so a single failed customer workflow can be traced end-to-end across every hop, external call, and retry.
- CloudWatch Synthetics canaries continuously exercise “golden path” workflows — for example, simulating a bank-file pull end-to-end — so failures are caught by synthetic monitoring before a real customer's scheduled job hits them.
- Alerting routes through EventBridge to SNS to PagerDuty/Slack, with severity tied to business impact: a Tier-1 customer's failed workflow pages on-call immediately, while a low-priority internal maintenance-job failure lands in a Slack channel — keeping signal-to-noise high.

8.5 Disaster Recovery



Disaster Recovery - Multi-Region Warm Standby

Figure 4 — Multi-region warm-standby disaster recovery.

The DR strategy is multi-region warm standby for Tier 1 and Tier 2 workloads (illustrated here as primary us-east-1 / DR us-west-2; region selection is a placeholder pending GTreasury's actual footprint), and backup-and-restore for Tier 3. Aurora Global Database replicates to the DR region in typically under a second, and promoting the secondary to a standalone writer is a single API call. DynamoDB Global Tables run active-active by design, so execution-state data is already live in both regions with no promotion step. S3 Cross-Region Replication keeps every raw and transformed file mirrored continuously. Every piece of infrastructure — Step Functions state machines, Lambda/Fargate task definitions, networking — is defined in infrastructure as code and deployed to both regions from the same pipeline, so the DR region is never a hand-maintained snapshot; it is redeployed on every release

and stays warm. Route 53 health-check-based failover, backed by Global Accelerator, redirects traffic automatically once the primary region's health checks fail a defined threshold. Critically, DR is exercised, not just documented: a scheduled quarterly failover game day actually cuts traffic to the DR region for a defined window and validates the runbook.

Tier	Example Workloads	RPO Target	RTO Target	Strategy
Tier 1	Business-critical, bank cut-off-driven customer workflows	≤ 1 minute	≤ 15 minutes	Warm standby; Aurora Global Database + DynamoDB Global Tables active-active
Tier 2	Standard customer workflows	≤ 15 minutes	≤ 1 hour	Warm standby with lower pre-provisioned DR-region capacity
Tier 3	Internal / reporting / batch jobs	≤ 24 hours	≤ 4 hours	Backup and restore; automated snapshots + on-demand IaC redeploy

8.6 AI-Augmented Operations & Developer Productivity

The case study explicitly asks that operational excellence “be balanced with innovations to improve developer productivity continually.” The AI agentic automation layer introduced in section 6.6 is this analysis's direct answer to that balance, touching both sides of it at once:

- **On the operations side**, the Ops/Recovery Copilot absorbs the lowest-risk, highest-volume share of failure triage automatically — the “operations timing out” and “connectivity issues” patterns called out in the case study — so human responders spend their attention on genuinely novel failures instead of repeating the same redrive action by hand every time.
- **On the developer-productivity side**, the Component Scaffolding Copilot removes the boilerplate cost of contributing a new catalog component (handler stub, schema, tests, IAM policy), which matters specifically because this architecture depends on components being contributed by many different GTreasury engineering teams, not just the platform team.
- **Guardrail by design, not by afterthought**: both copilots are scoped to a fixed, auditable tool set from day one (section 6.6), so the productivity gain never comes at the expense of the fault-tolerance and resiliency priorities that top the case study's own list — an under-scoped agent is a reliability risk, not a reliability win.

9. Security & Compliance Considerations

Because the platform moves financial data across many banking institutions and ERPs on behalf of many customers, security is treated as a baseline requirement of the architecture rather than a layer added afterward:

- Encryption in transit (TLS everywhere) and at rest (AWS KMS customer-managed keys across S3, Aurora, DynamoDB, and Secrets Manager).
- Least-privilege IAM scoped per component, enforced automatically at registration time by the Component Registry's CI/CD pipeline, so a contributed component can only reach the specific S3 prefixes, secrets, and tables it declares it needs.
- Full audit trail via AWS CloudTrail and immutable, versioned S3 execution logs — important both for incident forensics and for demonstrating control to customers and auditors.
- Centralized, automatically rotated credentials via Secrets Manager for every bank/ERP integration, eliminating credentials embedded in code or configuration.
- Network segmentation via VPC private subnets; PrivateLink/VPC endpoints wherever a bank or ERP integration supports it, and AWS Direct Connect considered for the largest strategic customers/partners, to avoid traversing the public internet.
- Continuous security monitoring via AWS GuardDuty, Security Hub, and AWS Config, feeding the same alerting pipeline as operational incidents.
- Compliance alignment (assumed, pending confirmation with GTreasury's actual compliance program) with frameworks typical for financial-data platforms, such as SOC 2 Type II and relevant data-residency requirements for customers with data-sovereignty constraints.
- **AI agent governance:** the Ops/Recovery and Component Scaffolding copilots (section 6.6) only ever receive sanitized operational metadata — error codes, job/execution identifiers, non-sensitive log excerpts — never raw bank account numbers, transaction-level financial data, or customer PII; Bedrock's data-handling commitment that customer content is not used to train underlying models is treated as a baseline requirement, not a bonus; every agent action is scoped to an explicit tool allowlist enforced by Bedrock Guardrails, with anything high-risk gated on human approval; and every agent decision is logged to CloudTrail tagged distinctly from human-initiated actions, so the audit trail always shows who — or what — took a given action.

10. Presentation Q&A — “The Challenge”

This section answers each question posed in the case study's four presentation segments, in presentation-ready form, wearing the Engineering Manager, Architect, and Lead Engineer hats the case study asks for in turn.

10.1 Execution & Delivery (~10 minutes)

Q: How would you go about the capacity planning process, factoring in an imaginary team size of your choosing?

Assume a team of 8 engineers plus an Engineering Manager, partnered with one Product Manager, working against both the existing platform and the new one. Capacity planning starts from historical data, not guesswork: pull the last 2–3 quarters of keep-the-lights-on (KTLO) ticket volume, on-call load, and unplanned-work ratio for the current scheduled-jobs platform to establish a realistic baseline of how much capacity is already spoken for.

- Split capacity explicitly: roughly 60–70% of the team's capacity to the new platform, 20–30% to legacy KTLO/support, and a standing 10–15% interrupt buffer for production incidents — published so stakeholders see the trade-off, not just the roadmap.
- Size work with story points or T-shirt sizes calibrated against the team's own historical throughput (velocity), not generic estimates, and re-baseline every quarter as the team's actual delivery rate becomes clearer.
- Build in explicit ramp time for the 2–3 engineers rotating onto the new platform from legacy work — the first sprint after a rotation is deliberately planned at reduced capacity.
- Treat the plan as a hypothesis, not a commitment: revisit capacity assumptions at each milestone boundary (section 10.1's second answer) rather than only at the start of the project.

Q: Working with Product Management, how would you go about structuring the project into a series of deliverable milestones, and how would you monitor and communicate progress?

Structure the roadmap around the same two-phase strategy as the technical design (section 4), broken into milestones that each ship something real and observable rather than internal-only groundwork:

1. M0 — Discovery & architecture spike (2–3 weeks): validate the Component Contract and pick the first 2–3 pilot workflows with Product Management, based on customer/business risk and technical simplicity.
2. M1 — Phase 1 MVP live (6–8 weeks): single-instance platform running the first pilot workflows in parallel with legacy, for a small trusted customer cohort.
3. M2 — Dual-run & validation (4–6 weeks): pilot workflows run on both platforms simultaneously; results are diffed to build confidence before legacy is turned off for those workflows.
4. M3 — Component catalog & CI/CD certification pipeline (6 weeks): opens the door for other engineering teams to contribute components.
5. M4 — Enterprise hardening (6–8 weeks): multi-AZ, autoscaling, full observability stack, alerting.
6. M5 — Multi-region DR & self-service recovery portal (8 weeks).
7. M6 — AI agentic automation layer (6–8 weeks): Ops/Recovery Copilot and Component Scaffolding Copilot, sequenced after the catalog and observability foundations they depend on are already in production.
8. M7 — Progressive migration & legacy decommission (ongoing, workflow by workflow, prioritized by risk and business value).

Progress is monitored and communicated through: bi-weekly sprint demos showing working software, not slides; a public Now/Next/Later roadmap Product Management and customers can see; a migration dashboard tracking percentage of workflows and customers moved; a RAID log (risks, assumptions, issues, dependencies) reviewed at a monthly steering committee with stakeholders; and a changelog that ties every shipped increment back to a milestone.

Q: Talk through your team composition and timelines knowing that the existing platform needs to be maintained while working on the next generation.

The 8-engineer team is organized as two pods with a rotation between them, not a permanent split, so knowledge of the legacy platform doesn't calcify in one or two people:

- **Platform pod (5–6 engineers, including 1–2 senior/staff):** owns the new orchestration platform build — Step Functions control plane, component catalog, CI/CD certification pipeline, observability.
- **Legacy/KTLO pod (2–3 engineers, rotating every 4–6 weeks):** keeps the existing scheduled-jobs platform stable and responds to production incidents, feeding lessons learned (common failure patterns, tricky bank integrations) directly back into the new platform's design.
- **1 SRE/DevOps-leaning engineer (can rotate through either pod):** owns CI/CD, IaC, and the observability stack across both platforms so tooling isn't duplicated.

The AI agentic automation layer (M6) is deliberately not a separate, permanent sub-team: 1–2 engineers from the Platform pod pick it up once the catalog and observability milestones are live, specifically because building the agent's Action Groups is largely wrapping the same Step Functions/Lambda APIs the platform pod already owns, not a distinct discipline requiring new headcount.

Indicative timeline: Phase 1 pilot live in ~8 weeks; enterprise-grade Phase 2 (multi-AZ, catalog, observability) at roughly 6–9 months; multi-region DR and full self-service recovery by ~9–12 months; the AI agentic layer 2–3 months after that, once it has real operational history to be grounded in; full legacy migration continuing on a workflow-by-workflow basis throughout, prioritized by customer risk rather than forced to a single cutover date.

Q: How do you show steady progress to your stakeholders on the work to further the vision that will benefit them while modernizing the platform?

- Track two parallel metric sets so “we're still stable” and “we're modernizing” are both visible: legacy platform health (uptime, incident count, MTTR) alongside new-platform adoption (% of workflows migrated, % of customers on new platform, self-service recovery usage rate).
- Demo working software every sprint, on real (pilot) customer data wherever possible, not mockups — the fastest way to build stakeholder trust in an infrastructure-heavy initiative.
- Share concrete before/after evidence as workflows migrate: a workflow's mean-time-to-recovery or a specific incident that would have paged Platform Operations under the old model but was self-served under the new one.
- Keep the roadmap visibly current (Now/Next/Later, reviewed monthly) so stakeholders see the plan evolve with reality rather than a static Gantt chart that quietly falls out of date.

10.2 Operational Excellence (~5 minutes)

Q: What principles would you implement to ensure the operational excellence of the system?

- **Reliability by design:** run AWS Well-Architected Framework reviews at each major milestone, not once at the end.

- **Automate everything:** infrastructure as code, CI/CD, and automated remediation wherever a failure pattern repeats more than once.
- **Define SLOs and error budgets:** use a burned error budget as the explicit trigger that shifts the team's priority from features to reliability, so that trade-off is data-driven, not a debate.
- **Blameless postmortems:** every incident produces action items that improve the system, never a search for who to blame.
- **Self-service tooling and runbooks:** if Platform Operations solves the same class of problem twice by hand, the third time should be a self-service action or an automated remediation.
- **Shift-left observability:** every new component ships with metrics, structured logs, and tracing built in — instrumentation is part of the definition of done, not bolted on after an incident.
- **Practice failure deliberately:** quarterly chaos/game days validate that resiliency and DR designs actually work under real failure conditions.
- **FinOps discipline:** cost per workflow/tenant is a first-class metric, so elastic scalability doesn't quietly become an unbounded cost.
- **Automate with an agent, not just a script, once a pattern is well understood:** the AI agentic automation layer (section 6.6) is the next rung up from static runbooks — once a failure pattern is well-documented and low-risk, the Ops/Recovery Copilot handles it directly instead of a human following a runbook by hand, under the same guardrail and audit discipline as every other automated action.

Q: What KPIs would you use to measure and drive improvements in operational excellence?

A small set of KPIs, each tied to a specific decision it should drive:

KPI	What It Drives
Job / workflow success rate (%)	Overall platform health; a leading indicator customers feel directly.
SLA adherence (% completed within window)	Whether time-sensitive jobs (e.g., bank cut-offs) are meeting business commitments.
Mean time to detect (MTTD)	Observability effectiveness — are we finding problems before customers report them?
Mean time to recover (MTTR)	Resiliency and runbook/self-service effectiveness.
Self-service recovery rate (% of incidents resolved without Platform Ops/Engineering)	Direct measure of the case study's self-service recovery goal.
Agent-resolved incident rate (% resolved by the AI agent with no human involvement)	Whether the agentic automation layer is actually absorbing toil, not just adding another dashboard.
Agent recommendation acceptance rate (for proposed, human-approved actions)	Whether the agent's higher-risk recommendations are trustworthy enough to be approved as-is.
Error budget consumption	Drives the feature-work vs. reliability-work trade-off objectively.
Alert-to-action ratio (actionable alerts / total alerts)	Alert fatigue and on-call sustainability.
Deployment frequency & change failure rate	Engineering excellence and delivery health (DORA metrics).
DR test success rate & cadence	Whether disaster recovery is proven, not just documented.
Cost per job / per tenant	Whether elastic scalability is staying cost-efficient at growing volume.

10.3 Engineering Excellence (~5 minutes)

Q: What are some of the best engineering excellence practices you will deploy to make this platform robust in terms of quality and incident management?

- **A full testing pyramid:** unit tests for component logic, integration tests against real (sandboxed) bank/ERP endpoints, and contract tests for the Component Contract itself, since many different teams will implement that contract independently.
- Automated static and security scanning (SAST/DAST, dependency scanning) gating every merge, and required for a component to be certified into the catalog.
- **Progressive delivery:** canary or blue/green deployments for both workflows (via versioned Step Functions aliases) and components, so a bad change affects a small percentage of traffic before it affects everyone.
- A defined incident severity matrix and on-call rotation with clear escalation paths, paired with blameless postmortems that produce tracked action items.
- Infrastructure and components both defined as code and reviewed through the same pull-request process as application code, so infrastructure changes get the same scrutiny.
- **AI-assisted scaffolding, human-reviewed merges:** the Component Scaffolding Copilot (section 6.6) generates a new component's boilerplate — handler, schema, tests, IAM stub — as a draft pull request, but every merge still goes through the same human code review and CI gates as any other change; the agent raises the floor on starting quality, it doesn't bypass review.

Q: How will you ensure the overall development life cycle is held at a high bar across the platform?

- A lightweight RFC/design-review process before any new component or workflow pattern is built, so architectural consistency doesn't depend on tribal knowledge.
- A single, enforced Definition of Done that includes tests, observability instrumentation, and documentation — not just “code compiles.”
- The Component Registry's CI/CD certification pipeline acts as the actual governance mechanism for the multi-team contribution model: a component cannot enter the shared catalog without passing contract tests, security scans, and load-testing headroom checks, regardless of which team wrote it.
- Architecture Decision Records (ADRs) kept alongside the code, so the reasoning behind a choice (including the alternatives in section 7) is discoverable months later, not lost to attrition.
- A standing tech-debt allocation (roughly 15–20% of sprint capacity) so quality work is scheduled, not perpetually deprioritized behind features.
- Documentation-as-code (runbooks, architecture docs, and onboarding guides versioned with the platform) so operational knowledge scales with the team, not just with a few senior engineers.

10.4 Technical Approach (~10 minutes)

Wearing the Architect and Lead Engineer hat: the end-to-end system flow that ties together every section above.

9. A trigger fires — a schedule via EventBridge Scheduler, a file arriving via AWS Transfer Family, a webhook via API Gateway, or a manual re-run from the self-service portal.
10. EventBridge normalizes the trigger into a single event format and starts (or resumes, on redrive) the corresponding Step Functions execution, whose workflow definition is resolved from Aurora.

11. Step Functions walks the workflow's DAG, invoking catalog components — Lambda for scripted logic, Fargate for compiled/long-running logic — for each step, applying the Retry/Catch policy declared in that step's Component Contract.
12. Components pull or push files and records via Transfer Family or API Gateway, transform them using shared transformation libraries, apply business rules, and write results to S3, Aurora, or DynamoDB as appropriate to the data's shape and access pattern.
13. Every step emits structured events to EventBridge (job started/completed/failed), which drive both the notification service (SNS/SES to customers and internal subscribers) and the observability pipeline (CloudWatch metrics/logs, X-Ray traces), correlated by a single execution ID.
14. On failure, the message or step lands in a dead-letter queue or a failed Step Functions state with a structured error, which immediately invokes the Bedrock-based Ops/Recovery Copilot (section 6.6); grounded against the runbook/postmortem Knowledge Base, the agent either executes a low-risk fix autonomously (redrive, clear the DLQ message, notify the subscriber) or proposes a higher-risk action for one-click human approval.
15. Whatever the agent could not confidently resolve is surfaced in the self-service portal exactly as a manually-triaged failure would be, with the agent's reasoning attached as context; Customer Support (and, eventually, the customer directly) can still trigger a manual Redrive, and repeated or unresolved failures escalate to on-call Platform Operations through the severity-based alerting path — the agent is an additional, faster front door onto the same recovery path, not a replacement for it.

This flow is identical in shape between Phase 1 and Phase 2 — only the underlying engine and execution substrate change (Airflow/EC2 versus Step Functions/Lambda/Fargate), and only Phase 2 has the agentic triage step — which is what makes the phased delivery strategy in section 4 possible without a rewrite.

11. Delivery Roadmap & Milestones

Milestone	Duration	Outcome
M0 — Discovery & architecture spike	2–3 weeks	Component Contract validated; first pilot workflows selected with Product Management.
M1 — Phase 1 MVP live	6–8 weeks	Single-instance platform running pilot workflows for a small trusted customer cohort.
M2 — Dual-run & validation	4–6 weeks	Pilot workflows run on legacy and new platform in parallel; results reconciled before cutover.
M3 — Component catalog & CI/CD certification	6 weeks	Any GTreasury engineering team can contribute a certified component.
M4 — Enterprise hardening	6–8 weeks	Multi-AZ, autoscaling, full observability stack, and alerting in production.
M5 — Multi-region DR & self-service recovery portal	8 weeks	Warm-standby DR live and tested; customers/Support can self-serve recovery.
M6 — AI agentic automation layer	6–8 weeks	Ops/Recovery Copilot live on low-risk auto-remediation; Component Scaffolding Copilot available to contributing teams.
M7 — Progressive migration & legacy decommission	Ongoing	Workflows migrated by risk/value priority until legacy platform is retired.

M6 is sequenced deliberately after M3–M5, not in parallel with them: the agent's Knowledge Base needs a real body of runbooks, postmortems, and certified component contracts to be grounded in, and the case for autonomous action is strongest once the catalog, observability, and DR foundations it relies on are already proven in production.

12. Assumptions

The case study intentionally leaves several specifics open; this analysis makes the following explicit assumptions, which should be validated with GTreasury stakeholders before implementation begins:

- Team size of 8 engineers plus 1 Engineering Manager and 1 Product Manager, used only to make the capacity-planning answer concrete — the same principles apply at other team sizes.
- AWS is the target cloud, per the assignment; specific region names (us-east-1 / us-west-2) are illustrative placeholders, not a recommendation on data residency, which should follow GTreasury's actual customer footprint and compliance obligations.
- A generic SOC 2 Type II-style compliance posture is assumed given the financial-data domain; actual compliance requirements (PCI DSS applicability, regional data-residency rules, customer-specific contractual requirements) should be confirmed with GTreasury's compliance/legal function.
- Customer and workflow volumes are estimated qualitatively from the case study's own example (“a hundred bank accounts globally” per customer); actual capacity and cost planning should be re-validated against GTreasury's real production metrics.
- Choice of AWS CDK versus Terraform for infrastructure as code, and CloudWatch/X-Ray versus a third-party observability platform, should ultimately follow whatever tooling GTreasury's broader engineering organization has already standardized on, to avoid introducing a second toolchain.
- The AI agentic automation layer (section 6.6) assumes GTreasury's data-governance and model-risk functions approve the use of Amazon Bedrock foundation models over operational metadata; the specific underlying model, the exact low-risk action allowlist, and the threshold at which an action requires human approval are all intentionally left as configurable policy decisions for GTreasury's Platform Operations and compliance stakeholders, not fixed in this design.